RECONCILIATION FOR FAILED UPI PAYMENTS

TAT Guard

Every stuck rupee, on a clock.

It finds the customers a bank owes, puts the most urgent cases first, and proves what was done.

Team Hackathon

Harshit Aryan

Param Vansh Singh



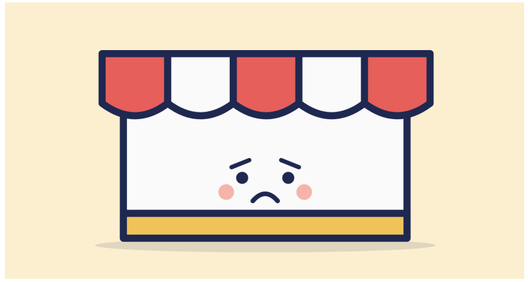
Built on Exasol Personal

All payment data is synthetic

Open source, MIT licence

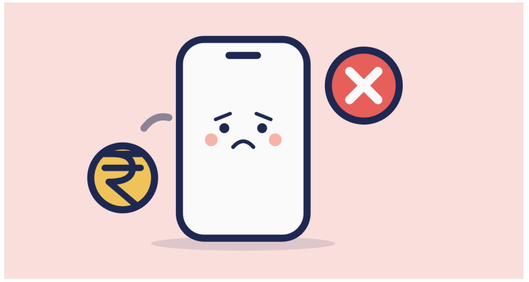
WHAT GOES WRONG

A failed payment can still take your money



1. A payment fails

A shopkeeper pays a supplier ₹2,000 on UPI. The app says failed.



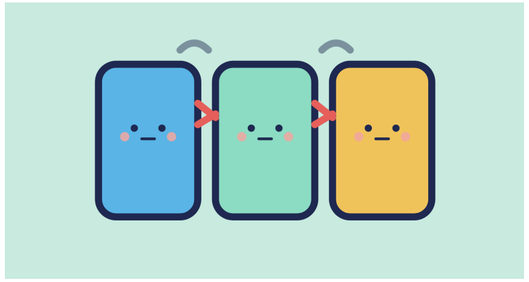
2. The money still left

The bank message says debited. The supplier never got it.



3. The clock runs

RBI sets a deadline. Past it, the bank owes ₹100 every day.



4. Nobody can see it

The proof sits in three systems, so often no one knows yet.

OUR PROBLEM STATEMENT

Bank reconciliation teams can’t see which failed UPI debits are about to start costing money, because the evidence is split across three systems that must be matched transaction by transaction. **If this worked,** they would see every stuck rupee on a clock, stop the cases that add cost tomorrow first, and show compensation was paid without the customer having to complain.

WHY IT MATTERS

The rule is clear, and the scale is enormous

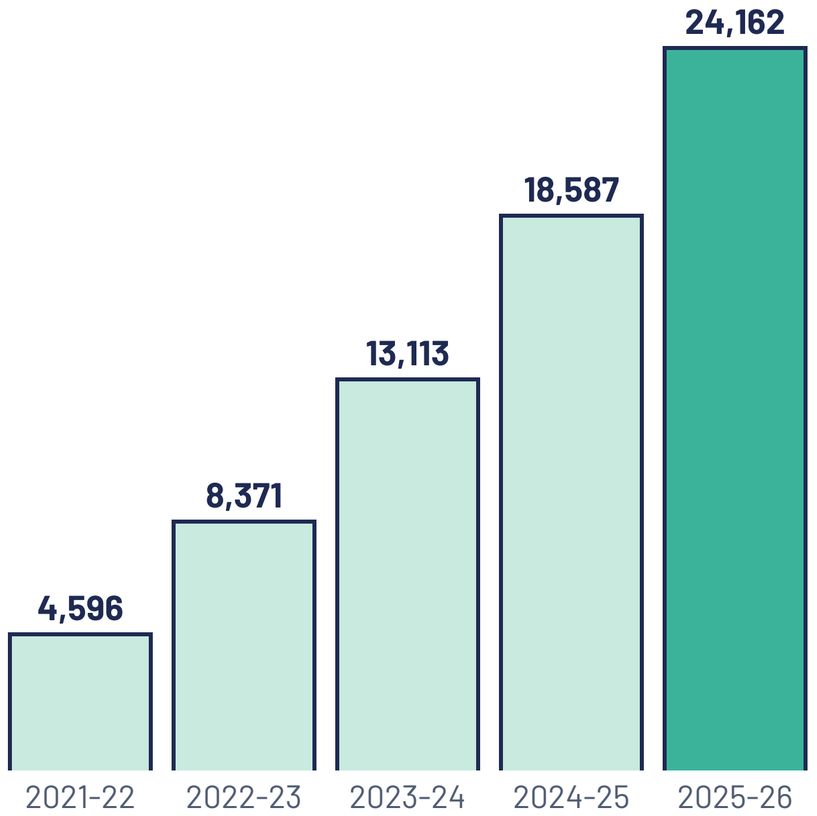
T+1
Fund transfer: reverse by the next calendar day

T+5
Merchant payment: reverse within five days

₹100
Owed to the customer for each day of delay

Suo moto
Paid without waiting for a complaint or a claim

UPI transactions a year, in crore



Daily average, Sept 2026
80.2 crore
UPI transactions a day

Complaints to RBI's ombudsman scheme
13.34 lakh
2024-25, up 13.55% from 11.75 lakh. All regulated entities, not only UPI.

RBI/2019-20/67 (no withdrawal found as of Oct 2026). Lok Sabha reply via Outlook Money. NPCI data via StartupTalky (secondary). RBI Ombudsman report 2024-25.

WHY IT STAYS HIDDEN

One payment leaves three different records



UPI switch log

What the bank's UPI system saw: success, failure or timeout.

`332833608267 ₹764.83`



NPCI settlement file

What settled, and in which cycle. Files can arrive late.

`332833608267 cycle 3`



Core banking ledger

What really left the account, with the reference in free text.

`UPI/DR/332833608267/MERCH…`



Too big for a sheet

Limit 10,48,576 rows; our sample has 28,93,031.



Cost stays invisible

A list of breaks does not show tomorrow's cost.



The customer chases

Notice, call the bank, wait, then the RBI portal.

Rows from our synthetic files; layouts are stand-ins, not real NPCI, switch or CBS formats. Sheet limit: Microsoft Excel specifications.

EXISTING WEB APPS

Good products exist. Compensation is the gap.

Product	UPI and NPCI matching	Disputes	Stage	₹100-a-day compensation	Proof paid unprompted
FSS Recon AI	Advertised	Advertised	Commercial	No mention	No mention
ISG Reconciliation	Advertised	Advertised	Live at banks	No mention	No mention
IDBI Intech i-Recon	Advertised	Advertised	Commercial	No mention	No mention
Smartstream Smart Agents	No mention	AI exceptions	Live at a Tier 1 bank	No mention	No mention
TAT Guard	**Built**	Not yet	Prototype, synthetic data	**Built**	**Built**

Where they are ahead

They run at real banks, with dispute workflows we have not built.

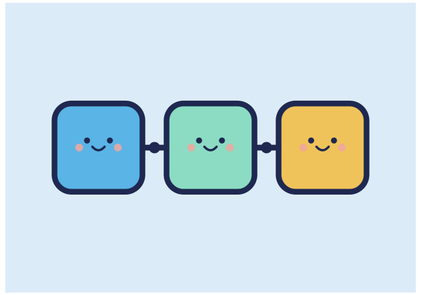
Where we add something

A queue built on the ₹100 clock, and proof it was paid unprompted.

"No mention" means not on the public page we read on 1 Nov 2026, not proof a feature is absent. ISG cites settlement penalties; i-Recon a T+1 TAT for its own reconciliation.

WHAT TAT GUARD DOES

One workbench, from a stuck rupee to proof



1. Match

Lines up switch, NPCI and ledger for every payment, inside Exasol.

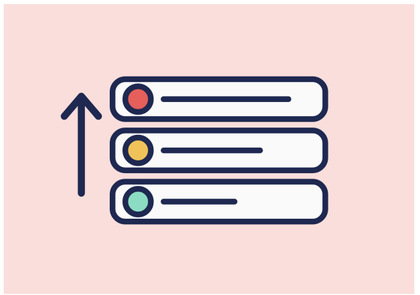
Four passes: id, RRN, Python UDF, fuzzy.



2. Start the clock

Finds who was debited, not refunded, then applies the deadline.

TAT days and ₹ a day load from facts.yaml.



3. Rank the work

₹100 a day is flat, so it ranks what adds cost tomorrow, then longest wait.

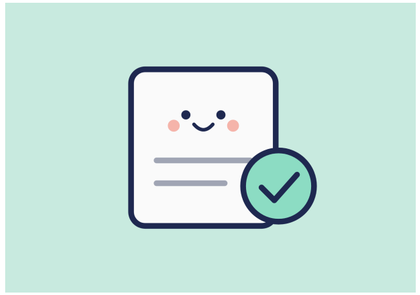
What accrued is sunk; tomorrow is not.



4. Explain

An assistant answers from read-only SQL and shows every query.

Every number is checked against results.



5. Approve, prove

A named person approves. SQL and before-after go in the audit log.

Nobody approves their own proposal.

WHO IT HELPS, AND WHAT THEY GET

Analyst: which case next, and why

Compliance: a register of what was paid

Ops lead: tomorrow's cost and the chase list

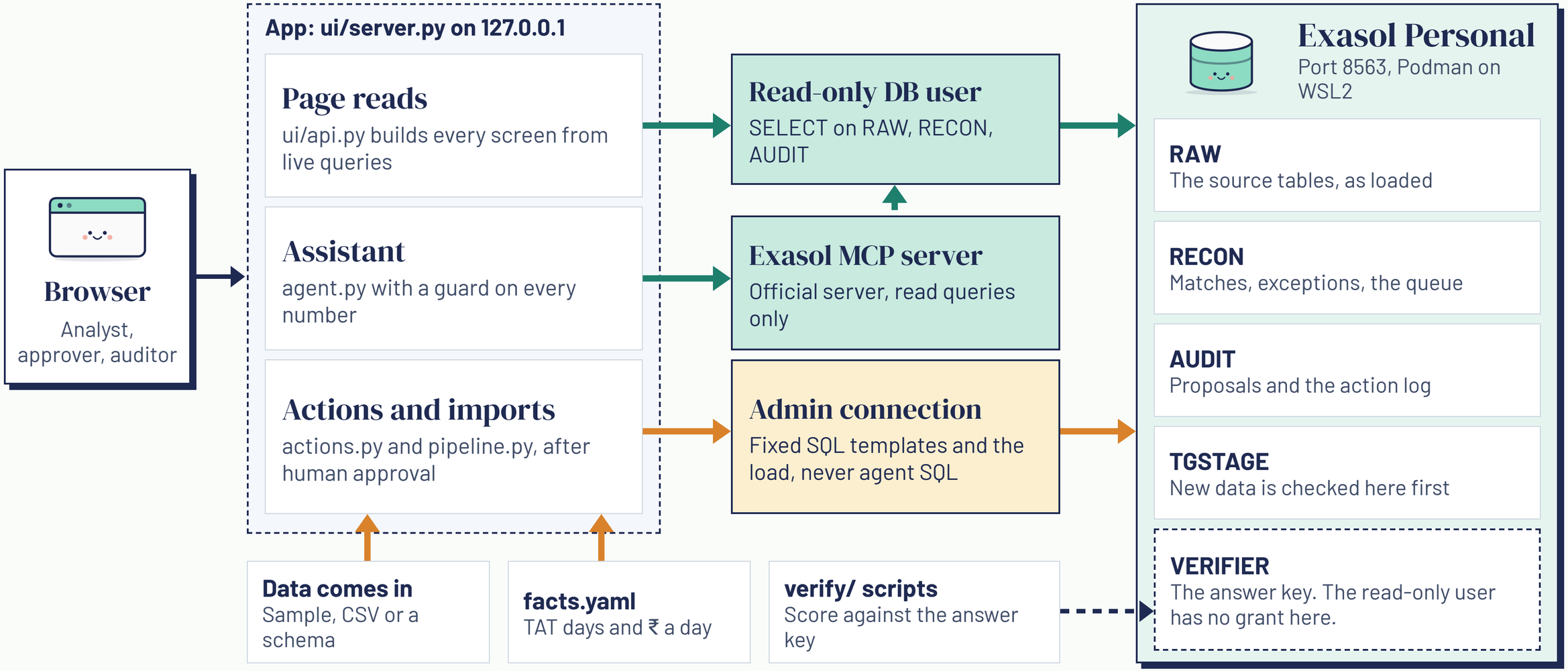
Tech team: open SQL they can check

Approver: exactly what will change

Customer: no need to complain first

THE WHOLE SYSTEM

Reads and writes never share a path



Runs on one machine. The assistant here uses fixed SQL playbooks; the Gemini path is written and tested offline only.

HOW WE ACCESS DATA

A number's journey, and who may touch it



Who asks	Goes through	Database access	What stops mistakes
Every screen	ui/api.py, live SQL	Read-only user	SELECT only; no figure is stored in the page
Assistant	agent.py to Exasol MCP server	Read-only user	Every number must appear in the query results
Approved fix	actions.py, then recon re-run	Admin connection	Named approver, fixed template, audit log
Import	pipeline.py; uploads via TGSTAGE	Admin connection	A bad file leaves the loaded data alone
verify/ scripts	Score results against VERIFIER	Admin connection	The read-only user cannot see the answer key

The agent can only create proposals. Three action types exist, and whoever proposes an action cannot approve it.

HOW THE DATA GETS CORRECTED

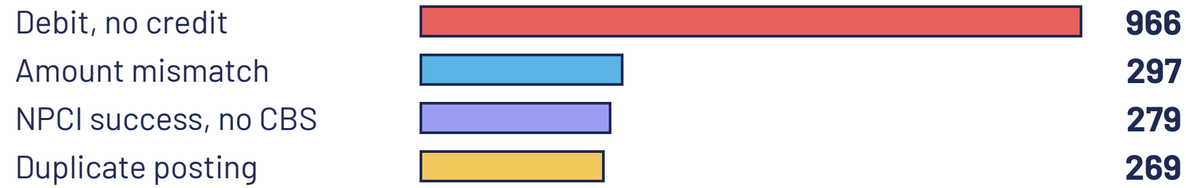
Four passes match it. The late file clears the noise.

1	**Transaction id** Switch to NPCI, id and amount	9,50,424
2	**RRN and time window** For rows that lost their id	1,901
3	**Python UDF in Exasol** Reads the reference from ledger text	9,40,128
4	**Fuzzy scoring** Amount, bank and time, no reference	578

Exceptions, before the late file 46,618

After the file arrives 1,811

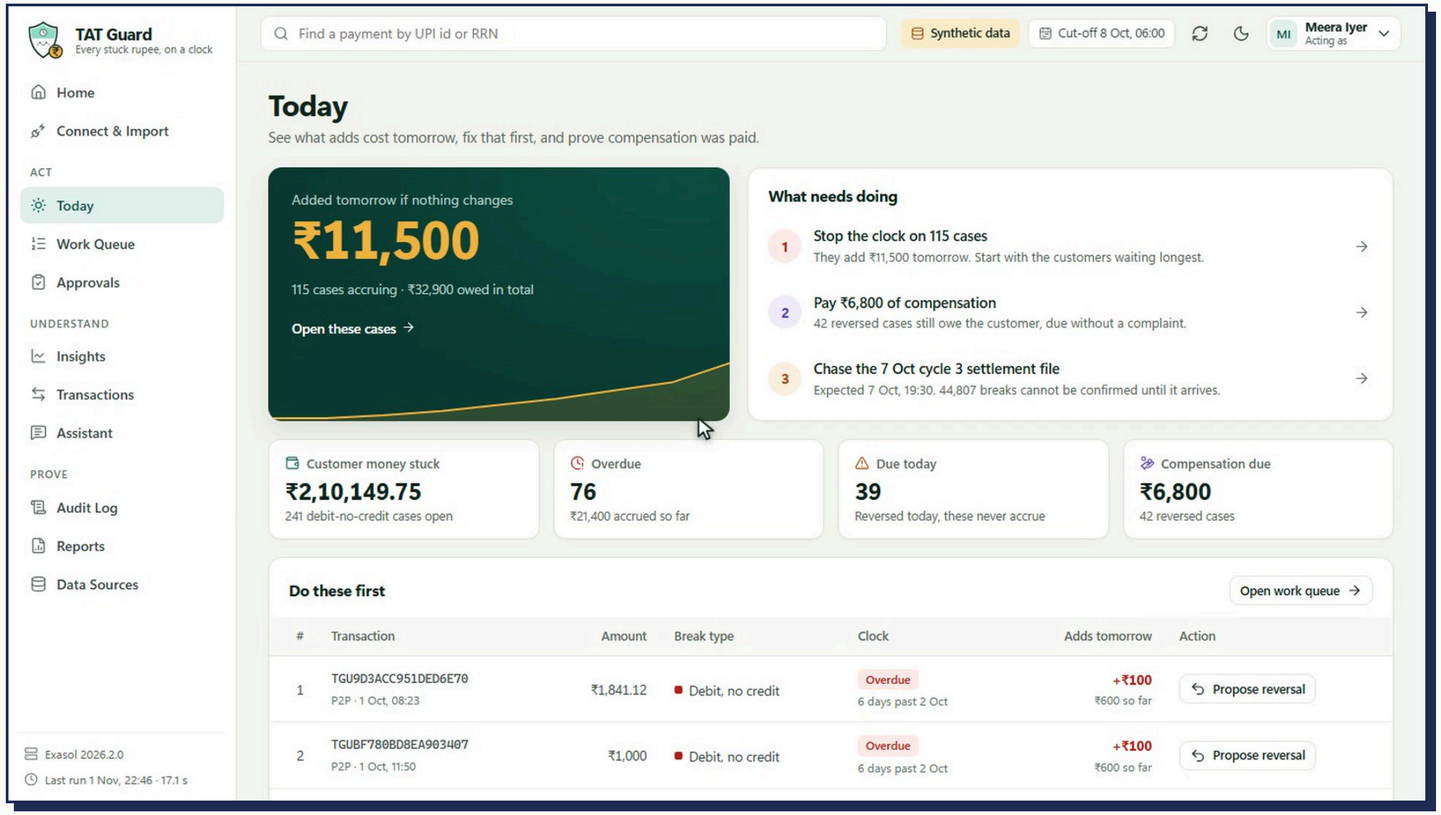
44,807 were only waiting for the file. Real breaks remain:



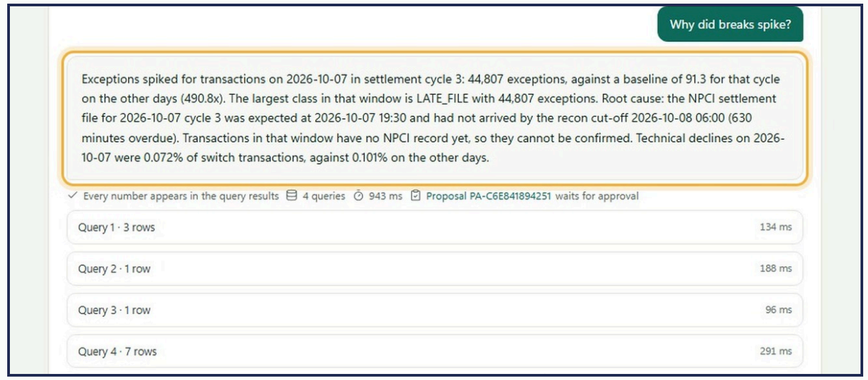
10,00,000 synthetic transactions, measured 1 Nov 2026. Passes 1 and 2 count switch rows; 3 and 4, ledger entries. Our generator held the late file back on purpose.

WHAT THE TEAM SEES

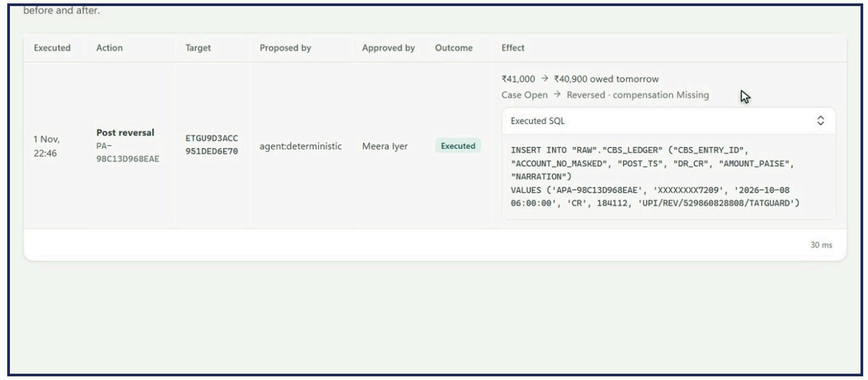
Live figures, a clear answer, and a full record



Today: ₹11,500 added tomorrow from 115 cases, ₹2,10,149.75 stuck, ₹6,800 compensation due.



Assistant: names the file that never arrived.



Audit log: who, when, exact SQL, before and after.

Screens from the live app: 10,00,000 synthetic transactions, cut-off 8 Oct 2026. Names in the app are made up.

TRADITIONAL WAY VS TAT GUARD

Same job, same laptop, same 46,618 exceptions



16.8 s bulk load plus 40.5 s reconciliation. One run of each, on one laptop; Exasol was limited to its VM's memory, pandas was not.

1.0
Precision and recall, all five classes, at 10 lakh rows

99.996%
469,007 of 469,024 at 1 crore rows; 3 wrong matches

0
Disagreements with an independent pandas matcher

14 of 14
Edge cases passed through the real SQL on Exasol

Synthetic data: this shows it finds the breaks our generator injects, not real-world accuracy. The 1 crore run predates the 31 Oct SQL changes.

WHO GAINS

The bank finds the case first

With this at a bank, people stop chasing their own money. Compensation is paid, and proved, without a complaint.



If 0.01% of 80.2 crore daily payments got stuck, and 5% of those ran a day late:

₹4.0 lakh a day, ₹14.6 crore a year

Illustrative: both rates are our assumptions, not data.

Today

A working prototype on synthetic data, not deployed at a bank.

Stated limits

Stand-in file formats, no login yet, and who is paid is an assumption.

Next

Real file formats, a real login, the dispute lifecycle, live Gemini.

Every stuck rupee, on a clock.

github.com/PROSTLE/TAT-Guard

MIT licence. An independent project, not affiliated with Exasol AG, NPCI or RBI.